

claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_b05b902c-a7c\agent-a3b4ad9493904df84.jsonl`  
- SHA-256: `526f1d306d8e1d44d56f939bf21be13bfe1ac4cab60204a8ee2e17b4a0f8dcd9`  
- Source modified: `2026-06-19T16:28:48+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_b05b902c-a7c`  
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`
```

Transcript

1. user (2026-06-19T16:25:21.914Z)

You are measuring the prominent-court shuttle gate's quality win on ONE multicourt badminton golden clip, against human golden labels. Work in the repo root; run Python via Bash. Be numerically exact ? do NOT estimate.

DATA

```
- Golden rallies: json.load(open("output/GX010128_golden_features.json"))["gts"] -> list of [start,end] seconds.  
- Manifest output/human_lovo_manifest.json: the entry with name==GX010128 gives trajectory (path), fps, frame_width (=1920 for these; so frame_height=1080).
```

STEP 1 baseline (ungated) local rallies:

```
from backend.eval.rally_seg_eval import windows_for  
from backend.eval.windowing import SERVED  
video={"name":NAME,"trajectory":TRAJ,"fps":FPS,"frame_width":FW,"gts":GTS}  
baseline = windows_for(video, SERVED)
```

STEP 2 derive a normalized (0-1) prominent-court polygon, RECALL-SAFE:

```
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner  
points = TrackNetRunner.parse_trajectory_csv(TRAJ) # each has .visible, .x, .y, .frame  
fh = 1080.0
```

Inspect the 2D distribution of visible (x/FW, y/FH). Multicourt clips can have multiple shuttle clusters (courts); the PROMINENT court = the dominant/foreground cluster (most shuttle mass; foreground tends lower/larger in frame). Draw a polygon (quad or bounding box in normalized coords) around the prominent cluster, EXCLUDING any clearly-separated secondary (adjacent) cluster.

VALIDATE recall-safety: for each golden rally compute its shuttle MEDIAN (x_norm,y_norm) over frames in [s*FPS, e*FPS]; your polygon MUST contain ≥ 0.85 of golden-rally medians (point_in_polygon from backend.pipeline.detectors.fusion_features). If < 0.85 widen it and re-check; iterate. Record pct_golden_in_polygon. If the clip has NO clear secondary cluster to exclude (courts overlap fully), say so ? a near-full-frame polygon is then a near no-op (an honest null).

STEP 3 gate + score (numerically exact):

```
from backend.pipeline.detectors.court_gate import gate_intervals_to_court  
gated = gate_intervals_to_court(baseline, points, polygon, FPS, FW, fh, min_in_court_frac=0.5)  
from backend.eval.metrics import score, match_intervals  
from backend.eval.tolerance_metrics import match_1to1  
b = score(baseline, GTS, 0.5); g = score(gated, GTS, 0.5) # .precision/.recall/.f1  
base_tolf1 = match_1to1(baseline, GTS)[3]; gated_tolf1 = match_1to1(gated, GTS)[3]
```

Of the DROPPED rallies (baseline minus gated), count how many were baseline FALSE POSITIVES: a baseline pred is a FP if it is NOT in match_intervals(baseline, GTS, 0.5).matches as a matched pred index.

```
seg_ratio = n_pred / n_gt.
```

STEP 4 return the structured row. The win is REAL only if precision/F1 RISE with recall ~FLAT (you cut FPs, not real rallies). If recall drops materially, or the clip barely over-fires, or courts don't separate -> say so honestly (a null/negative result is a valid, valuable finding).
pct_golden_in_polygon < 0.85 means your polygon is unsafe ? fix it before reporting.

YOUR CLIP: GX010128

2. assistant (2026-06-19T16:25:24.446Z)

I'll work through this step by step. Let me start by exploring the data and relevant code.

3. user (2026-06-19T16:25:26.685Z)

```
keys: ['name', 'fps', 'frame_width', 'candidates', 'gts']
gts: [[9.661, 32.173], [40.419, 47.165], [53.165, 57.672], [65.18, 73.675], [81.922, 85.925],
[91.674, 98.674], [103.428, 106.431], [115.169, 120.921], [128.815, 135.826], [140.32, 142.819],
[148.821, 153.574], [160.825, 164.816], [171.339, 174.828], [182.816, 196.067], [202.065,
213.564], [221.077, 226.821], [234.576, 239.061], [246.815, 255.572], [264.566, 269.325],
[312.985, 315.737], [322.731, 326.733], [333.233, 341.234], [347.738, 353.989], [362.484,
372.237], [383.486, 390.232], [400.483, 403.237], [410.484, 416.486], [431.737, 440.23],
[449.218, 451.731], [455.806, 459.552], [465.553, 472.306], [480.807, 486.556], [491.801,
493.809], [504.807, 517.81], [523.806, 530.054], [538.052, 543.307], [550.06, 554.555],
[560.826, 568.326], [577.763, 582.012], [589.511, 596.263], [603.76, 609.765], [616.262,
620.257], [625.511, 629.763], [646.011, 649.762], [656.762, 660.008], [666.51, 669.262],
[676.758, 680.255], [687.007, 692.01], [696.76, 702.759], [708.56, 717.065], [722.749, 728.251],
[736.003, 749.009]]
```

4. user (2026-06-19T16:25:28.118Z)

```
<class 'list'>
```

5. assistant (2026-06-19T16:25:30.608Z)

The golden set has 53 rallies. Let me look at the manifest entry for GX010128 and the relevant code modules.

6. user (2026-06-19T16:25:33.332Z)

```
{'name': 'GX010128', 'fps': 59.94, 'frame_width': 1920.0, 'labels':
'C:/Users/avidu/Projects/badminton-highlight-indexer/output/GX010128.rallies.csv', 'trajectory':
'C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/GX010128_june12_traj.csv'}
trajectory: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX010128_june12_traj.csv
fps: 59.94
frame_width: 1920.0
```

7. user (2026-06-19T16:25:33.602Z)

```
1      """Prominent-court shuttle gate ? multicourt G2 (see
  ``docs/PROMINENT_COURT_DETECTION.md``).
2
3      In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
  on a
4      background / adjacent court becomes a false rally on the filmed match. This gate re-
  defines the
5      shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
  a visible
6      shuttle point whose normalised position is outside the prominent-court polygon is marked
  NOT
7      visible, so the velocity windower never opens a rally there.
8
9      Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10     ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
  visibility, never
```

```

11     set it ? so on a correct polygon it suppresses out-of-court rallies without
manufacturing any.
12     ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
court /
13     unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
14
15     The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
16     ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
a point up
17     to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
still
18     admitted, so a high clear over the prominent court is not clipped.
19     """
20     from __future__ import annotations
21
22     from dataclasses import replace
23     from typing import Any, List, Optional, Sequence, Tuple
24
25     from backend.pipeline.detectors.fusion_features import point_in_polygon
26
27     Polygon = Sequence[Tuple[float, float]]
28
29
30     def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
31         """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?
32
33         The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
34         finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
35         over the prominent court, admit it."""
36         if point_in_polygon((nx, ny), polygon):
37             return True
38         return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)
39
40
41     def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
42                             fps: float, frame_width: float, frame_height: float,
43                             height_pad: float = 0.0) -> float:
44         """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.
45
46         ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
47         visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
48         leave that judgement to the windower)."""
49         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
50             return 1.0
51         fs, fe = interval[0] * fps, interval[1] * fps
52         n = inside = 0
53         for p in points:
54             if getattr(p, "visible", False) and fs <= p.frame <= fe:
55                 n += 1
56                 if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
57                     inside += 1
58         return 1.0 if n == 0 else inside / n
59
60
61     def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
62                                 polygon: Optional[Polygon], fps: float,
63                                 frame_width: float, frame_height: float,

```

```

64             min_in_court_frac: float = 0.5, height_pad: float = 0.0
65             ) -> List[Tuple[float, float]]:
66         """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.
67
68         Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
69         occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
70         window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard
71         point gate). This drops an interval whose in-court point fraction is below
`min_in_court_frac`,
72         so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
(returns all
73         intervals) when `polygon` is None/degenerate. Subtractive-only."""
74         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
75             return intervals
76         return [iv for iv in intervals
77                 if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
78                     >= min_in_court_frac]
79
80
81     def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
82                             frame_width: float, frame_height: float,
83                             height_pad: float = 0.0) -> List[Any]:
84         """Return `points` with visibility CLEARED for any visible shuttle outside the
prominent-court
85         polygon (expanded upward by `height_pad`).
86
87         No-op (returns the input list unchanged) when `polygon` is None/degenerate or the
frame dims
88         are non-positive. Otherwise each gated point is a NEW `TrajectoryPoint` (the input
list and
89         its points are never mutated)."""
90         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
91             return points
92         out: List[Any] = []
93         for p in points:
94             if getattr(p, "visible", False):
95                 nx, ny = p.x / frame_width, p.y / frame_height
96                 if not admitted(nx, ny, polygon, height_pad):
97                     p = replace(p, visible=False)
98             out.append(p)
99         return out
100

```

8. assistant (2026-06-19T16:26:08.107Z)

Note: the manifest entry doesn't have `gts` or `frame\_width` keyed exactly as the prompt described ? `frame\_width` is there (1920.0), but `gts` is in the golden_features file. Let me check the key modules' signatures.

9. user (2026-06-19T16:26:10.210Z)

```

windows_for sig: (video: 'Dict[str, Any]', preset: 'WindowingPreset', min_crossings: 'int' = 0)
-> 'List[Interval]'
SERVED: WindowingPreset(name='served', velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=2.0, max_window_duration=6.0, hard_cap=False, inactivity_end=1.5,
inactivity_rally_thresh=0.2, inactivity_min_density=0.3, blob_fallback=False, blob_factor=4.0)
<class 'backend.eval.windowing.WindowingPreset'>

```

10. user (2026-06-19T16:26:12.294Z)

```
parse_trajectory_csv: (csv_win_path: str) ->
List[backend.pipeline.detectors.tracknet_runner.TrajectoryPoint]
point_in_polygon: (point: 'Tuple[float, float]', polygon: 'Sequence[Tuple[float, float]]') ->
'bool'
score: (preds: List[Tuple[float, float]], gts: List[Tuple[float, float]], iou_threshold: float =
0.5, match_result: Optional[backend.eval.metrics.MatchResult] = None) ->
backend.eval.metrics.Score
match_intervals: (preds: List[Tuple[float, float]], gts: List[Tuple[float, float]],
iou_threshold: float = 0.5) -> backend.eval.metrics.MatchResult
match_1to1: (preds: 'List[Interval]', gts: 'List[Interval]', tau_start: 'float' = 2.0, tau_end:
'float' = 1.5) -> 'Tuple[List[Match], float, float, float]'
```

11. assistant (2026-06-19T16:26:15.229Z)

Let me check `windows\_for`'s requirements on the video dict, the `MatchResult` structure, the `TrajectoryPoint` fields, and confirm the trajectory path exists.

12. user (2026-06-19T16:26:17.103Z)

```
def windows_for(video: Dict[str, Any], preset: WindowingPreset,
                min_crossings: int = 0) -> List[Interval]:
    """Parse a video's trajectory, window it under ``preset`` ? predicted rally intervals.

    ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
    ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than this
    many
    times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only, derived
    from the trajectory + an auto-estimated net axis; it cleans up the over-split that bounded
    windowing (W1) leaves behind.

    NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate would
    drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate is
    strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no harm".
    """
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
    wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
    ivs = windowing.windows_to_intervals(wins)
    # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
    normalized
    # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the prominent
    court ? a
    # background/adjacent-court rally. Window-level (not point-level): a background rally's
    central
    # tail keeps a few in-court points, enough to hold a window, so point-clearing alone is too
    leaky
    # (measured on GX010142 rally #1). No polygon ? no-op. See
    docs/PROMINENT_COURT_DETECTION.md.
    poly = video.get("court_polygon")
    if poly and ivs:
        from backend.pipeline.detectors.court_gate import gate_intervals_to_court
        fw = float(video["frame_width"])
        fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
        ivs = gate_intervals_to_court(
            ivs, points, poly, float(video["fps"]), fw, fh,
            min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
            height_pad=float(video.get("court_height_pad", 0.0) or 0.0))
    if min_crossings > 0 and ivs:
        from backend.pipeline.detectors.rally_gate import gate_windows
        gated = [(g.start, g.end)
                  for g in gate_windows(points, ivs, video["fps"], min_crossings=min_crossings)
                  if g.kept]
    # Gating emptied a non-empty video ? keep the ungated windows.
```

```
ivs = gated if gated else ivs
return ivs
```

13. user (2026-06-19T16:26:19.769Z)

```
MatchResult fields: dict_keys(['matches', 'unmatched_pred_indices', 'unmatched_gt_indices'])
TrajectoryPoint fields: dict_keys(['frame', 'visible', 'x', 'y'])
traj exists: True
```

14. assistant (2026-06-19T16:26:22.830Z)

Now let me run STEP 1 (baseline) and inspect the trajectory point distribution for STEP 2.

15. user (2026-06-19T16:26:29.254Z)

```
n_gt: 52
baseline n_pred: 81
baseline first 5: [(0.016683350016683352, 4.237570904237571), (7.574240907574241,
10.026693360026695), (10.56056056056056, 14.78144811478145), (15.298631965298632,
19.035702369035704), (19.636302969636304, 24.691358024691358)]
total points: 45298
visible points: 27133
frame range: 0 45297
x range: 83.76044511795044 1680.000114440918
y range: 3.2102923840284348 758.9580917358398
```

16. assistant (2026-06-19T16:26:33.659Z)

Baseline has 81 predictions against 52 golden rallies (seg_ratio ~1.56, materially over-firing ? good candidate). Note: y max is 758, well below 1080 ? shuttle is concentrated in the upper portion of the frame. Let me build a 2D histogram of the normalized visible-shuttle distribution to identify court clusters.

17. user (2026-06-19T16:26:39.086Z)

```
n visible: 27133
rows=x bins (0..1 left->right), cols=y bins (0..1 top->bottom)
y: 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
x0.0-0.1: 0 0 0 2 0 61 86 0 0 0
x0.1-0.2: 0 0 2 11 21 148 83 0 0 0
x0.2-0.3: 9 67 62 150 54 182 28 0 0 0
x0.3-0.4: 63 179 151 304 251 183 32 0 0 0
x0.4-0.5: 88 234 585 1309 7663 218 2 0 0 0
x0.5-0.6: 306 425 850 3231 1458 698 24 1 0 0
x0.6-0.7: 91 176 869 1959 1507 1058 0 1 0 0
x0.7-0.8: 0 41 92 188 225 8 0 0 0 0
x0.8-0.9: 1 1 0 1688 7 0 0 0 0 0
x0.9-1.0: 0 0 0 0 0 0 0 0 0 0

x percentiles: [0.13600066 0.34182856 0.48715651 0.52567685 0.61408967 0.87118195
0.875 ]
y percentiles: [0.04969981 0.17487498 0.32653369 0.40714884 0.45668814 0.53102385
0.59802913]
```

18. assistant (2026-06-19T16:26:42.789Z)

There's a clear structure here. The dominant mass is in x?[0.4,0.7], y?[0.2,0.6]. But notice an anomalous spike at x0.8-0.9, y0.3-0.4 (1688 points) ? a separate cluster, likely a secondary/adjacent court. Let me look more closely at that region and at the high-x region overall.

19. user (2026-06-19T16:26:51.502Z)

```
points x>0.75: 1844
```

```

their x range: 0.7503643035888672 0.8750000596046448
their y range: 0.09228031502829658 0.5
x median: 0.8720344305038452 y median: 0.32620908154381645
finer hist x[0.75,1.0] y[0.2,0.5]:
x0.75:    7    12    42    70    12
x0.80:    0     0     0    10     0
x0.85:    0   299  1386     0     0
x0.90:    0     0     0     0     0
x0.95:    0     0     0     0     0
frames of x>0.8 cluster: min 58 max 45235 count 1697
points x in [0.7,0.8]: 554 y median 0.3703426122665405

```

20. assistant (2026-06-19T16:26:58.319Z)

There's a very tight, dense secondary cluster at $x \approx 0.85$ - 0.875 , $y \approx 0.30$ - 0.40 (~1685 points, all at $x \approx [0.85, 0.875]$). This is a degenerate-tight blob ? almost a single pixel location ? which strongly suggests a stuck/spurious tracker point or a distant adjacent court, not the prominent foreground match. The prominent court mass is $x \approx [0.4, 0.7]$, $y \approx [0.2, 0.6]$.

Now the critical recall-safety step: compute golden-rally shuttle medians and see which polygon contains ?85% of them. Let me compute golden medians first.

21. user (2026-06-19T16:27:05.983Z)

```

golden rallies with a median: 52 of 52
golden median x range: 0.33179566264152527 0.6922121644020081
golden median y range: 0.23846663369072807 0.5252839194403754
golden median x pctl5: [0.33179566 0.46392817 0.55889384 0.6454195 0.69221216]
golden median y pctl5: [0.23846663 0.32273527 0.36822498 0.45668795 0.52528392]
golden medians with x>0.78: 0
golden medians with x>0.72: 0

```

22. assistant (2026-06-19T16:27:11.400Z)

This is clean. All 52 golden-rally medians fall in $x \in [0.33, 0.69]$, $y \in [0.24, 0.53]$? entirely within the prominent foreground court, and NONE in the secondary $x \approx 0.85$ cluster. The secondary cluster ($x \approx 0.85$, ~1685 stuck points) is exactly what the gate should exclude.

Let me design a recall-safe polygon around the prominent court that excludes the $x > 0.78$ cluster, and validate it contains ?85% (here, all) of golden medians.

23. user (2026-06-19T16:27:19.654Z)

```

polygon: [(0.15, 0.0), (0.78, 0.0), (0.78, 0.7), (0.15, 0.7)]
golden medians inside: 52 / 52 = 1.0
visible pts inside polygon: 25063 / 27133 = 0.9237
visible pts EXCLUDED: 2070

```

24. assistant (2026-06-19T16:27:25.691Z)

The polygon ``[(0.15,0.0),(0.78,0.0),(0.78,0.70),(0.15,0.70)]`` contains 100% of golden medians (recall-safe, well above 0.85) and excludes 2070 visible points (the $x \approx 0.85$ secondary cluster). Now STEP 3: gate and score.

25. user (2026-06-19T16:27:39.351Z)

```

Exit code 1
Traceback (most recent call last):
  File "<string>", line 37, in <module>
    matched_pred_idx=set(p for (p,gt) in mr.matches)
  File "<string>", line 37, in <genexpr>
    matched_pred_idx=set(p for (p,gt) in mr.matches)
                                ^^^^^^^
TypeError: cannot unpack non-iterable Match object

```

```
n_gt: 52
BASE  n_pred 81 P 0.4938 R 0.7692 F1 0.6015 tolF1 0.4662
GATED n_pred 81 P 0.4938 R 0.7692 F1 0.6015 tolF1 0.4662
n_dropped: 0
```

26. assistant (2026-06-19T16:27:44.199Z)

Important finding: GATED == BASE ? n_dropped is 0. The gate dropped nothing. Let me confirm why, and fix the Match unpacking. The reason: the secondary cluster (x?0.85, ~1685 points) is temporally distributed, but none of the baseline windows had a *majority* of their visible points out-of-court. Let me verify by checking the in-court fraction of every baseline window.

27. user (2026-06-19T16:27:47.507Z)

```
Match type fields: inspect
[Match(pred_index=0, gt_index=0, iou=1.0)]
@dataclass
class Match:
    """One matched predicted?ground-truth pair."""
    pred_index: int
    gt_index: int
    iou: float
```

28. user (2026-06-19T16:27:55.060Z)

```
in-court fraction stats over 81 baseline windows:
min 0.6231 p5 0.7705 p25 0.92 median 0.9771
windows with frac<0.5: 0
windows with frac<0.9: 18
lowest 5 fracs: [np.float64(0.6231), np.float64(0.6581), np.float64(0.7273),
np.float64(0.7694), np.float64(0.7705)]
```

29. assistant (2026-06-19T16:28:02.346Z)

This is the honest result. The minimum in-court fraction across all 81 baseline windows is 0.6231 ? no window is majority out-of-court, so at `min\_in\_court\_frac=0.5` the gate drops nothing. The secondary x?0.85 cluster's ~1685 points are real but scattered thinly across many windows (never dominating any single one), and likely co-occur with in-court points so no window crosses below 0.5.

This is a near-null result for this clip at the standard threshold. Let me confirm the gate is a true no-op (precision/recall/F1 all flat) using the correct Match field, and double-check whether even a stricter threshold would help (for the honest read).

30. user (2026-06-19T16:28:20.808Z)

```
=== RESULTS GX010128 ===
n_gt 52
BASE  n_pred 81 P 0.493827 R 0.769231 F1 0.601504 tolF1 0.466165
GATED n_pred 81 P 0.493827 R 0.769231 F1 0.601504 tolF1 0.466165
baseline FPs total: 41
n_dropped 0 n_dropped_were_fp 0
dP 0.0 dR 0.0 dF1 0.0
thr=0.6: n_pred=81 dropped=0 dropped_were_fp=0 P=0.4938 R=0.7692 F1=0.6015
thr=0.7: n_pred=79 dropped=2 dropped_were_fp=2 P=0.5063 R=0.7692 F1=0.6107
thr=0.75: n_pred=78 dropped=3 dropped_were_fp=2 P=0.5 R=0.75 F1=0.6
thr=0.8: n_pred=75 dropped=6 dropped_were_fp=3 P=0.4933 R=0.7115 F1=0.5827
```

31. assistant (2026-06-19T16:28:30.911Z)

The picture is now complete and honest. At the specified `min\_in\_court\_frac=0.5`, the gate is an exact no-op on GX010128: nothing dropped, precision/recall/F1 all identical.

Key findings:

- The clip DOES have a clearly-separated secondary cluster ($x \approx 0.85$, $y \approx 0.30-0.40$, ~1685 tightly-stuck shuttle points ? an adjacent/background court or a stuck tracker), and the polygon correctly excludes it (2070 visible points removed, 100% of golden medians retained ? fully recall-safe).
- BUT no baseline window is majority-out-of-court: the minimum in-court fraction across all 81 windows is 0.6231, well above the 0.5 gate threshold. The out-of-court points are sprinkled thinly across windows, never dominating one. So at the standard threshold the gate cuts zero false positives here.
- This is a genuine null for GX010128 at the production threshold ? not because courts fail to separate (they do separate cleanly), but because the secondary-court points never accumulate into a *majority-out* window. (Stricter thresholds like 0.7 would drop 2 windows, both FPs, for +0.009 F1 ? but that is not the configured gate and risks recall at 0.8.)

Baseline is heavily over-fired (81 preds vs 52 golden, seg_ratio 1.56, 41 FPs), so there was room to win ? the gate just doesn't capture these particular FPs at min_frac=0.5.

32. user (2026-06-19T16:28:39.702Z)

Structured output provided successfully

33. assistant (2026-06-19T16:28:48.752Z)

Completed the analysis for GX010128. Result is an honest null at the production threshold.

Summary:

- The clip clearly has two shuttle clusters: a dominant prominent-court mass ($x \in [0.4, 0.7]$, $y \in [0.2, 0.6]$) and a tightly-stuck secondary cluster (~1685 points at $x \approx 0.85$, $y \approx 0.30-0.40$? an adjacent court or stuck tracker).
- The derived polygon ``[(0.15,0.0),(0.78,0.0),(0.78,0.70),(0.15,0.70)]`` is recall-safe: it contains 100% of the 52 golden-rally shuttle medians while excluding 2070 visible secondary-cluster points.
- At the configured ``min_in_court_frac=0.5`` the gate drops nothing ? the minimum in-court fraction over all 81 baseline windows is 0.6231, so no window is majority-out-of-court. Precision/recall/F1 are all unchanged (0.4938 / 0.7692 / 0.6015).
- Baseline over-fires heavily (81 preds vs 52 golden, 41 FPs), so there was headroom, but these particular FPs are not out-of-court-majority. Stricter thresholds (0.7) would clip 2 FP windows for a negligible +0.009 F1, but that is not the production config and risks recall at 0.8.

Relevant files:

- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\court_gate.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\output\GX010128_golden_features.json``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\output\human_lovo_manifest.json``
- Trajectory: ``C:\Users\avidu\Projects\Annotation Setup\Collect\Trajectories\GX010128_june12_traj.csv``